

What to fix first, and who can fix it

Measured live on 23 September 2026 from Copenhagen, through the read-only WooCommerce API and public requests. No admin or server access was used.

Pages that come from the cache load fast, including ad clicks. The slow part is everything a cache can't serve: search, cart and checkout. Each of those waits 1.3–2 seconds for WordPress to start, and 5 seconds or more when a few people shop at the same time. Some of the 8 September suggestions are done (13 plugins removed). The ones that target this slowness directly are still open.

Contents

- | | |
|--|--|
| 1. Start here | 7. Handing it to Claude Code |
| 2. Where the site is hosted | 8. Corrections to the 8 September report |
| 3. Server capacity: what we measured | 9. Limits |
| 4. Where things stand | 10. Sources |
| 5. Issues ranked by sales impact | 11. Appendix: runbook |
| 6. Who can do what | |

Start here: three steps, in this order

- 1. This week, in wp-admin (about an hour with Claude Code):** stop the background cart refresh (issue 3) and the fake account sign-ups (issue 5). The setup is under [Handing it to Claude Code](#), and every change can be undone with one click.
- 2. Send the IT admin a short message** asking for a few server numbers and for your own access to Cloudways and Cloudflare. Claude drafts it for you in the last runbook task, and you decide whether to send it.
- 3. With those numbers, agree on the server side together:** PHP workers or CPU cores (issue 1) and the search fix (issue 2). These two matter most for sales.

Where the site is hosted

The shop's own WooCommerce system report puts its files at `/home/1447285.cloudwaysapps.com/...` . That folder layout exists only on **Cloudways**, a managed host that rents cloud servers from DigitalOcean, Vultr, AWS or Google. **Cloudflare** handles the domain's DNS and sits in front of the server. The exact server setup isn't visible from outside, so the next section asks the admin for a few numbers.

Server capacity: what we measured

The IT admin says the machine has 64 GB of RAM and the site uses about 13 GB. That's good news: memory is clearly not a problem, and nothing here suggests adding more. The open question is a different one: how many requests the site can process *at the same time*. That depends on CPU cores and PHP workers rather than RAM.

To get a feel for it from the outside, we sent the same very light request (WordPress starts and answers without doing anything else) once, then 2, 3, 4, 6 and 8 at a time:

SENT AT ONCE	FASTEST	SLOWEST	REQUESTS HANDLED IN PARALLEL
1	1.54 s	1.54 s	1
2	1.43 s	1.73 s	≈ 1.7
3	1.40 s	2.55 s	≈ 1.6
4	1.49 s	2.96 s	≈ 1.9
6	1.49 s	3.87 s	≈ 2.2
8	1.32 s	3.62 s	≈ 3.1

"Handled in parallel" = requests sent × 1.4 s of work each ÷ time until the last one finished. One run on 23 Sep 2026, with the shop's normal traffic running at the same time. It's an outside estimate, and the server's own numbers are more reliable.

The requests start taking turns at about 2–3 at a time. That can have several ordinary causes: a conservative PHP-FPM worker limit, a CPU limit on the container, or something in the PHP code

that makes requests wait for each other. The admin can see which one it is in a minute, and we can't from the outside.

What would help us: the output of these standard commands from the server (or container), so we can line our measurements up with the real configuration:

```
nproc                                # CPU cores available to the site
free -g                             # memory total / used
systemd-detect-virt                 # virtualisation type
cat /sys/class/dmi/id/sys_vendor     # hardware / cloud provider
grep -rh "max_children" /etc/php/*/fpm/pool.d/ 2>/dev/null # PHP-FPM worker
limit
uptime                              # load average
```

What we'd look for together: if the worker limit or the available cores is around 2–4, raising it is probably the quickest win, and that's a settings change, not a new machine. If both are already high, the time is going somewhere in the PHP code, and the Query Monitor pass in Task 3 should point to it.

Where things stand (23 Sep)

1.3–1.6 s

WordPress startup on every uncached request. Healthy is 0.15–0.3 s. Was 1.46 s on 8 Sep.

1.8–2.1 s

Every search. It is never cached. One search took 14 s today.

5.3 s

Slowest of 6 searches sent at the same time

40 → 27

Active plugins. Removing 13 didn't change startup time.

Ranked by how much each issue costs in sales

Ordered by how close the problem sits to the purchase, how many shoppers hit it, and what Baymard found about shopper behaviour. Baymard doesn't publish its own speed-to-conversion figures (see [Limits](#)), so the speed link comes from a Google-commissioned Deloitte study: a 0.1 s faster mobile site went with **8.4% more retail conversions**. That study shows a correlation, not proof of cause.

1

CRITICAL

Cart and checkout wait on the server at every step

WHAT SHOPPERS
FEEL

The cart page takes 1.7 s. The checkout request takes 1.3 s even when it only redirects an empty cart. Checkout updates and "place order" are the same kind of request as the measured 1.3 s cart-refresh call, so they pay the same startup time. With 6 people active at once, requests wait up to 5 s.

WHY IT RANKS FIRST

Every buyer passes through here, and none of it can ever be cached. In Baymard's abandonment survey, **17%** of shoppers who abandoned a checkout said the *"website had errors / crashed"*. That is what stalls and timeouts look like to a shopper.

FIX

Think of a shop with too few cashiers, each of them slow. It needs two separate fixes, and they multiply.

More cashiers (IT admin, in Cloudways): the server handles only 2–3 requests at a time (see [server capacity](#)). Raise PHP workers under Application Settings → PHP-FPM Settings (`pm.max_children`). If the CPU cores are already busy, add cores with Vertical Scaling. Memory already looks ample. This can't be done from wp-admin.

Faster cashiers (owner, in wp-admin): cut the 1.4 s of work each request does, with runbook Tasks 1–4. Task 3 shows where that time actually goes, because removing plugins by guesswork hasn't worked.

WHO

IT admin: Cloudways

Owner: wp-admin

2

CRITICAL

Search takes about 2 seconds, every time

WHAT SHOPPERS
FEEL

1.8–2.1 s per search, 5 s at busy moments. The page cache never serves search results.

WHY

Searchers know what they want. Baymard found that when search disappoints, shoppers *"quickly conclude that a site didn't carry the product"*. It also found 37% of sites fail **symptom searches**, meaning problem-based queries. Pest control is almost all problem searches: *mus i loftet, myrer i køkkenet*. Baymard measured whether search finds the right product, not how fast it is. A slow search makes that failure worse.

FIX

A Cloudflare cache rule is the quick try, but it may not work: search pages set cookies, and Cloudflare doesn't cache responses that set cookies. It has to be tested. The fix that will hold is an instant search that runs in the browser from a small index of the 205 products, with synonyms for problem searches. That is about a day of development work.

WHO

IT admin: Cloudflare

IT admin: code

3

HIGH

A background cart refresh on every visitor's first page

WHAT SHOPPERS
FEEL

Nothing directly, because it runs in the background. But each new visitor holds a server worker for 1.3–3 s just to refresh the mini-cart dropdown. That is the same capacity checkout needs at busy times, so it feeds issue 1.

FIX

One snippet pasted into Code Snippets (runbook Task 1). About 10 minutes, and undone with one click. This is the first thing to do, because it is the easiest.

WHO

Owner: wp-admin

4

MEDIUM

Sorting, filters and pages that fall out of the cache

WHAT SHOPPERS
FEEL

Sorting the shop by price takes 1.7 s. A product page not in the cache takes 2.6 s, against 0.15–0.25 s when it is. Good news: ad and campaign clicks (`?gcLid=` , `?utm_...`) are served from the cache in 0.14–0.4 s.

FIX

No separate job. It improves along with issue 1.

WHO

Follows from #1

5

MEDIUM

84,165 user accounts for about 13,400 orders

WHAT SHOPPERS
FEEL

Nothing directly. The user tables nearly doubled in 4 weeks (user data went from 88 to 167 MB), which points to bot sign-ups on the My Account page. That hasn't been confirmed yet (Task 5 checks).

KEEP

Guest checkout is on, and it should stay on. **18%** of shoppers who abandon checkout say it was because *"the site wanted me to create an account"* (Baymard).

FIX

Turn on reCAPTCHA for registration, or turn off sign-up on the My Account page (runbook Task 5).

WHO

Owner: wp-admin

6

LOW

Language markers leak into Google's product data

WHAT SHOPPERS
FEEL

Nothing on the site itself. The structured data Google reads carries a product description like `[ :da]Fang skægkræ...[ :sv]...[ : ]` , which can make Google's snippets for the shop look messy.

WHO

IT admin

7 Who can do what

LOW

About 620 MB of leftover database

tables, faster order storage (HPOS) still off

The owner can do issues 3 and 5 alone, and gather the facts for issue 1. The rest needs

Cloudways or Cloudflare access, which only the IT admin has today. The other option is having their own login to both, since it's their shop and their account.

ACCESS	WHAT IT UNLOCKS	WHO HAS IT NOW
wp-admin	Cart-refresh snippet, autoload check, Query Monitor measurement, removing leftover plugins, stopping bot sign-ups	Owner
Cloudways panel	Real server size (settles the 64 GB claim), PHP workers, OPcache, backups, staging copy, database cleanup	IT admin
Cloudflare	Search cache rule	IT admin

Handing it to Claude Code

Claude Code can drive the owner's own Chrome through the **Claude in Chrome** extension. The owner logs in to wp-admin, and Claude clicks through the changes while the owner watches and approves each one. Claude also runs the speed measurements in its terminal, so every change gets a before and after number. It never sees the password.

1. Install [Claude Code](#) and the [Claude in Chrome extension](#). It needs a Pro, Max, Team or Enterprise plan, signed in with `/login`.
2. Log in to **billigskadedyr.dk/wp-admin** in Chrome.
3. Make an empty folder, open a terminal in it, and start Claude Code with Chrome turned on:

```
claude --chrome
```
4. Open the runbook file attached to this PDF (paperclip / attachments panel), or select all the text in [the appendix](#). Paste it into Claude Code and press Enter.
5. Answer "yes" when Claude asks before each change. It stops after every task and reports the numbers.

The runbook's safety rules: one change at a time, a backup confirmed before starting, an instant undo if any page errors, no orders, updates, payment settings or SQL, and customer data described as patterns only. The last task drafts the message to the IT admin for everything the owner can't do alone.

Corrections to the 8 September report

- **Cart refresh was called "blocking". It isn't.** The request runs in the background after the page shows. Its real cost is server capacity (issue 3), not a frozen page.
- **The suggested Cloudflare rule can't be built as written on most plans.** Keying the cache on only the `s` parameter is an Enterprise feature. The logged-in bypass has to go in the rule's condition instead. Search responses also set cookies and send `max-age=0`, so the rule may not cache at all. Test it before relying on it.
- **The language-marker leak moved.** It's no longer visible on the product page. It now sits only in the structured data for Google.

Limits

- Baymard doesn't publish its own numbers linking speed to conversion. The 40%-leave-after-3-seconds figure on its statistics page is quoted from Toptal. This page uses Baymard only for what it measured itself: why shoppers abandon checkout, and how search fails.
- Each Baymard reason is the share of shoppers who abandoned a checkout, not a measured loss for this shop. Baymard's full methodology for these figures is paywalled.
- The Deloitte 8.4% figure covers mobile retail sites across Europe and the US over 4 weeks in 2020. It's a correlation, commissioned by Google.
- Checkout was measured with an empty cart and no order was placed. The "every checkout step pays startup" point is inferred from the measured background cart request, which is the same type of request.
- Server size, PHP worker count and OPcache weren't checked, because the Cloudways panel isn't reachable from outside.
- Single timings vary by about ± 0.3 s. The 14 s search was a one-off. Four spaced repeats afterwards took 1.8–2.1 s.

Sources

- [Baymard: Cart abandonment rate statistics](#). Abandonment reasons: errors/crashed 17%, account required 18%, too long/complicated 17%.
- [Baymard: Ecommerce search query types](#). Symptom searches fail on 37% of sites; 56% of sites have mediocre or worse search (updated 29 Apr 2026).
- [Baymard: E-commerce search usability report](#)
- [Deloitte for Google: Milliseconds Make Millions \(2020\)](#)
- [Cloudflare: Cache Rules settings](#) and [default cache behaviour](#)
- [Claude Code: Use Claude Code with Chrome](#)

Appendix: runbook for Claude Code

The same text is attached to this PDF as **billigskadedyr-runbook.md**.

```
# billigskadedyr.dk speed runbook (for Claude Code)
```

Prepared 23 September 2026. Paste this whole file into Claude Code, started with `claude --chrome`, and it will walk the shop owner through the fixes one at a time.

```
## Your role, Claude
```

You are helping the owner of billigskadedyr.dk, a WooCommerce shop, make the site faster. The owner has WordPress admin access but no server, database, hosting-panel or Cloudflare access, and is not a programmer. You work through their Chrome browser (Claude in Chrome) and run `curl` measurements yourself in the terminal.

Save this text as `RUNBOOK.md` in the current folder so you can re-read it, then start with Task 0.

```
## Rules (follow all of them)
```

1. The owner logs in to wp-admin themselves. Never ask for, type or store a password.
2. One change at a time. Before any change, say in one sentence what you will change and how to undo it, then wait for the owner to say yes.
3. After every change: run the measurement below, then run the shop check below. Report the before/after numbers in one short table.
4. If any page shows an error, a white screen or "critical error": immediately undo the last change (deactivate the snippet or plugin, or revert the setting), confirm the site is back, and stop.
5. If wp-admin itself stops loading, stop and tell the owner to call the IT admin. Do not try anything else.
6. Never: place a real order, change payment/shipping settings, update WordPress/WooCommerce/plugins, delete users, run SQL, or edit theme files.
7. Customer data: when you look at users or orders, describe patterns only. Do not copy names, emails, addresses or phone numbers into the chat or any file.
8. Do not touch these plugins (they carry payments, pricing, shipping, tracking or caching): WooCommerce, ePay Payment Solutions, WooCommerce EAN Betalings Metode, WooCommerce Tiered Price Table, UDFASET - Shipmondo, Weight Based Shipping, Emaerket, Partner-ads, Redirection, Breeze, Object Cache Pro, Code Snippets, qTranslate-XT (deactivating it crashed the site on 8 Sep 2026), Yoast SEO, XT Ajax Add To Cart, Checkout Field Editor, Sequential Order Numbers, Lager og profit tabel, Google Ads API Server Side, WP Consent API, Cookie Popup, reCAPTCHA for WooCommerce, CURCY Multi Currency (business decision, see end).

```
## Measurement (run in the terminal, spaced, never in bursts)
```

```
```bash
for i in 1 2 3; do curl -s -o /dev/null -w 'startup %{time_starttransfer}s\n'
'https://billigskadedyr.dk/wp-admin/admin-ajax.php?action=zzz_nonexistent'; sleep 3;
done
for i in 1 2 3; do curl -s -o /dev/null -w 'search %{time_starttransfer}s\n'
'https://billigskadedyr.dk/?s=rotte'; sleep 3; done
```
```

"startup" is the time WordPress needs to start before it does any work. Every search,

cart and checkout request pays it. Baseline on 23 Sep 2026: startup 1.30-1.64 s, search 1.8-2.1 s. A healthy WooCommerce site starts in 0.15-0.30 s. Single runs vary by ± 0.3 s, so compare the 3-run averages.

Shop check (in Chrome, after every change)

Open the home page, one product page, and a search for "mus". Add one product to the cart and confirm the mini-cart updates. Open /kurv/ and confirm the item is there. Open /kasse/ and confirm the checkout form loads (do not submit it). Remove the item from the cart.

Task 0: Baseline and backup

1. Run the measurement and show the owner the numbers.
2. Ask the owner: "Has the IT admin confirmed there is a backup from the last 24 hours?" Cloudways makes automatic backups, but only the IT admin can see them. Do not start Task 1 without a yes.

Task 1: Stop the cart-refresh request on normal pages (about 10 minutes)

Why: every new visitor triggers a background request (`?wc-ajax=get\_refreshed\_fragments`) that holds a server worker for 1.3-3 s just to refresh the mini-cart dropdown. That capacity is what cart and checkout need at busy times.

1. Check first: `curl -s "https://billigskadedyr.dk/?nocache=\$RANDOM" | grep -c cart-fragments`. The result should be above 0.
2. wp-admin → Snippets → Add New.
3. Title: `Perf: dequeue wc-cart-fragments (except cart/checkout/account)`
4. Paste this code, with NO ``<?php` line at the top:`

```
`php
add_action( 'wp_enqueue_scripts', function () {
    // WooCommerce off → is_cart() undefined → white screen. Guard it.
    if ( ! function_exists( 'is_cart' ) ) {
        return;
    }
    if ( is_cart() || is_checkout() || is_account_page() ) {
        return;
    }
    // dequeue, never deregister - XT Ajax Add To Cart declares this as a
    dependency
    wp_dequeue_script( 'wc-cart-fragments' );
}, 20 );
`
```

5. Scope: "Only run on site front-end". Click "Save Changes and Activate".
6. Clear the page cache: in the wp-admin top bar, Breeze → Purge All Cache.
7. Verify: repeat the step 1 curl. The result must now be `0`. Then run the shop check. Adding to cart must still update the mini-cart. XT Ajax Add To Cart returns its own fragments, so it should.
8. Known side effect: a returning visitor who already has items in the cart sees an empty mini-cart dropdown until they open /kurv/. That is acceptable.
9. Undo: Snippets → deactivate this snippet.

Task 2: Check the autoloaded options (read-only, 2 minutes)

1. wp-admin → Tools → Site Health → Status.
2. Look for "Autoloaded options could affect performance". If it is there, report the size it states. Under 800 KB is healthy.
3. Change nothing. Add the result to the IT admin message (Task 6).

Task 3: Find out where the 1.4 s startup goes (about 20 minutes)

The 13 plugins removed since 8 September did not reduce startup time, so guessing which plugin to remove is not working. Measure instead.

1. Ask the owner to approve installing "Query Monitor" by John Blackbourn (Plugins → Add New). Install and activate it.
2. As the logged-in admin, open <https://billigskadedyr.dk/?s=rotte> and open the Query Monitor panel from the admin bar.
3. Write down: page generation time; the top 5 in "Queries by Component" (time per plugin); everything under "HTTP API Calls" (external calls made while the page loads are a classic cause of slow startup; note each URL's host and time); anything under "Slow Queries" and "Errors".
4. Repeat on /kurv/ with one item in the cart. Remove the item afterwards.
5. Deactivate and delete Query Monitor.
6. Put the findings in a short table for the owner and add them to the IT admin message.

Task 4: Remove leftover plugins, one at a time (about 15 minutes each)

After EACH one: run the measurement and the shop check, and report the result. If startup does not move, say so plainly. Some of these are cleanup rather than speed.

- a. User Role Editor: deactivate. Roles already set stay in place.
- b. ManageWP - Worker: first ask the owner whether ManageWP is used for backups or updates. If yes, or if unsure, skip it and add the question to the IT admin message.
- c. The two Google product feeds (CTX Feed and WooCommerce Google Product Feed): the owner opens Google Merchant Center → Products → Feeds (or Data sources) and reads which feed URL Google fetches. A URL containing `ctxfeed` belongs to CTX Feed. Deactivate only the plugin whose feed is not used. If unsure, skip it. Never touch Partner-ads, which has its own feed.

Task 5: Stop fake account sign-ups (about 15 minutes)

The shop has 84,165 user accounts but about 13,400 orders in total. That points to bot sign-ups.

1. Open `https://billigskadedyr.dk/wp-admin/users.php?orderby=registered&order=desc`. Look at the newest 50 accounts: registration times close together, random-looking usernames, no orders. Describe the pattern only (rule 7). Delete nothing.
2. If it looks like bots, let the owner choose one of these:
 - wp-admin → reCAPTCHA for WooCommerce settings → enable it on the My Account registration form, or
 - WooCommerce → Settings → Accounts & Privacy → untick "Allow customers to create an account on the My account page". Existing customers can still log in.
3. Keep "Allow customers to place orders without an account" ON. Forcing account creation is a well-documented reason shoppers abandon checkout.
4. Undo: revert the setting.

Task 6: Draft a message to the IT admin (the owner sends it)

Draft a short, polite message in Danish with these requests, and include the results from Tasks 2-5:

1. Give the owner team access to the Cloudways account, or at minimum a screenshot of Server Management showing the cloud provider, RAM and vCPU. The site's own system report shows it runs on Cloudways (server path `/home/1447285.cloudwaysapps.com/...`) behind Cloudflare.
2. The concern is CPU capacity, not RAM. A concurrency test on 23 Sep showed the server handles only about 2-3 requests at a time (8 simultaneous requests took up to 3.6 s, against 1.4 s for one). Ask the IT admin to paste the output of ``nproc`, `free -g`, `systemd-detect-virt`, `cat /sys/class/dmi/id/sys_vendor`, `grep -rh max_children /etc/php/*/fpm/pool.d/`` and ``uptime``, and to confirm OPcache is on. If cores or ``max_children`` are around 2-4, raise them. Adding RAM will not help.
3. Give the owner access to the Cloudflare account for the domain.
4. Confirm the backup schedule and set up a Cloudways staging copy for future changes.
5. Database cleanup from section 7 of the 8 Sep report: about 620 MB of tables left behind by removed plugins, and a leftover ``wp_`` table set.
6. qTranslate language markers (``[:da]...[:sv]...[::]``) leak into the product structured data (JSON-LD ``description``) that Google reads.
7. Search: see "Phase 2" below.

Show the draft to the owner. Do not send anything yourself.

Phase 2: needs the IT admin or Cloudflare access (not for this session)

- Search speed. Search can never use the page cache, and responses set cookies (``wmc_current_currency`` from CURCY, ``woocommerce_cart_hash``) plus ``cache-control: max-age=0``. A Cloudflare Cache Rule for ``?s=`` URLs may therefore not cache at all. It must be tested: the second identical request has to show ``cf-cache-status: HIT``. On non-Enterprise plans the rule cannot limit the cache key to the ``s`` parameter, and the "logged-in" bypass must be written into the rule expression (for example ``not http.cookie contains "wordpress_logged_in"``). The more robust fix is an instant search that runs in the browser from a small JSON index of the ~205 products, including synonyms for problem searches such as "mus i loftet" or "myrer i køkkenet". This is about a day of development work.
- CURCY Multi Currency: if the shop only sells in DKK, removing it drops a cookie that blocks caching and one plugin from every request. Business decision, test on staging.
- Server capacity (PHP workers / plan size): Cloudways panel only.